

PROJECT_SPEC.md

Expense Tracker — Product & Architecture Specification

1. Purpose

The Expense Tracker is a personal finance management application designed to help users move beyond simply recording transactions.

The long-term product direction is:

> Track → Understand → Plan → Predict → Coach → Act → Review → Improve

The product should help users answer:

1. Where am I financially?
2. Am I on track?
3. What should I do next?

The system should progressively evolve from an expense tracker into a personal financial management and decision-support platform.

2. Product Vision

The application should give users a clear understanding of their financial position without overwhelming them.

The product should combine:

- transaction tracking
- budgeting
- savings
- financial planning
- financial goals
- recurring obligations
- financial health indicators
- forecasting
- personalized insights
- AI-assisted explanations and guidance

The application should prioritize clarity and useful action over displaying large amounts of financial data.

3. Product Principles

3.1 Financial correctness

Financial calculations are core product functionality.

Balances, budgets, totals, forecasts, and spending limits must be calculated deterministically by the application.

AI must never be the authoritative source for financial calculations.

3.2 User trust

Users should be able to understand why the application is showing a financial number or recommendation.

Where practical, important calculations should be explainable.

Example:

```text

Safe to Spend

= Available Money

- Upcoming Obligations
- Planned Savings
- Reserved Amounts

```

The exact formula must be defined before implementation.

3.3 Progressive complexity

Do not expose every financial feature immediately.

The product should progressively introduce more advanced capabilities as the underlying data becomes available.

3.4 Simple first

Prefer the simplest architecture capable of supporting the current product requirements.

Do not build infrastructure for hypothetical future scale unless there is a clear technical reason.

4. Target User Problem

Many expense applications stop at:

> "Here is what you spent."

This product should go further:

> "Here is what happened, why it matters, what is likely to happen next, and what you can consider doing."

The system should therefore transform raw financial activity into useful context.

5. Product Capability Model

The product can be understood as several layers.

Layer 1 — Record

Capture:

- income
- expenses
- categories
- dates
- descriptions
- payment methods
- savings
- transfers where supported

Layer 2 — Understand

Provide:

- spending summaries
- income summaries
- category breakdowns
- monthly comparisons
- savings progress
- spending trends

Layer 3 — Plan

Support:

- monthly plans
- category budgets
- savings targets
- financial goals
- recurring bills
- planned expenses

Layer 4 — Predict

Estimate:

- end-of-month position
- budget risk
- upcoming obligations
- potential shortfalls
- projected savings
- spending trajectory

Predictions must clearly be presented as estimates rather than guaranteed outcomes.

Layer 5 — Coach

Provide contextual explanations and suggestions based on verified application data.

AI can help communicate insights but must not replace deterministic financial logic.

Layer 6 — Act

Help users take useful actions such as:

- adjusting a budget
- recording an upcoming expense
- increasing a savings target
- reviewing a spending category
- acknowledging a financial alert

Layer 7 — Review

Provide periodic reviews:

- weekly
- monthly
- custom periods where appropriate

Layer 8 — Improve

Use historical user data to help identify recurring financial behavior and patterns.

6. Core Existing Domain

The current application is centered around financial transactions.

Core concepts include:

- User
- Transaction
- Income
- Expense
- Category
- Savings
- Balance
- Monthly financial activity

The existing implementation is the source of truth for the exact current schema.

Before changing domain models, inspect the repository and database.

Do not assume this specification overrides the actual existing implementation.

7. Planned Domain Model

As the product evolves, additional domain concepts may include:

```text

User

■■■■ Transactions

■■■■ Budgets

■■■■ Goals

■■■■ Bills

■■■■ Savings Plans

■■■■ Monthly Plans

■■■■ Financial Reviews

■■■■ Alerts

■■■■ Financial Insights

```

Possible future entities:

Transaction

Represents a financial movement.

Potential fields:

- id
- userId
- type
- amount

- categoryId
- description
- date
- paymentMethod
- createdAt
- updatedAt

The actual schema must follow the current codebase.

Budget

Represents a planned spending limit.

Potential fields:

- id
- userId
- categoryId
- amount
- period
- startDate
- endDate

Goal

Represents a financial objective.

Potential fields:

- id
- userId
- name
- targetAmount
- currentAmount
- targetDate
- status

Bill

Represents an expected recurring or one-time obligation.

Potential fields:

- id
- userId
- name

- amount
- dueDate
- recurrence
- status

These are conceptual models, not instructions to create all of them immediately.

8. Monthly Financial Plan

The monthly plan should become one of the central product concepts.

A user should be able to define a plan for a specific month.

A plan may contain:

- expected income
- planned expenses
- category budgets
- savings allocation
- bills
- goals
- discretionary spending
- financial buffer

The system should compare:

```text

Planned

vs

Actual

vs

Projected

```

This comparison is more useful than displaying transactions alone.

9. Budgeting

Budgets should support category-level planning.

Example:

```text

Food ■80,000

Transport ■50,000

Utilities ■40,000

Shopping ■30,000

Savings ■100,000

...

The system should be able to calculate:

```text

Budget

• Actual Spending

= Remaining Budget

...

Where appropriate, it should also calculate:

```text

Remaining Budget

÷ Remaining Days

= Suggested Daily Budget

...

Any such calculation must account for the actual product rules before being implemented.

## 10. Safe-to-Spend

Safe-to-Spend is a major planned feature.

It should answer a question such as:

> "How much money can I reasonably spend without putting my current financial plan at risk?"

It should not simply equal:

```text

Balance - Expenses

...

Potential inputs include:

- current available balance
- expected income
- upcoming bills
- committed expenses
- planned savings

- financial goals
- budget allocations
- recurring obligations
- time remaining in the financial period

The exact formula must be designed and documented before implementation.

The result should be explainable.

Example conceptual output:

```
```text
```

```
Safe to Spend: ■45,000
```

Based on:

- Current available funds
- ■25,000 upcoming bill
- ■30,000 planned savings
- Remaining days in the month

```
```
```

This is an illustrative structure, not a finalized formula.

11. Financial Health

The product may eventually provide a financial health view.

Potential dimensions:

- spending stability
- budget adherence
- savings consistency
- emergency buffer
- recurring obligations
- income stability
- goal progress

Do not reduce financial health to an arbitrary single score without defining the methodology.

If a score is introduced, its inputs and interpretation must be documented.

12. Forecasting

Forecasting should use deterministic calculations or appropriately designed statistical/ML models.

Potential forecasts include:

- projected monthly spending
- projected savings
- budget exhaustion
- upcoming cash-flow pressure
- goal completion estimates

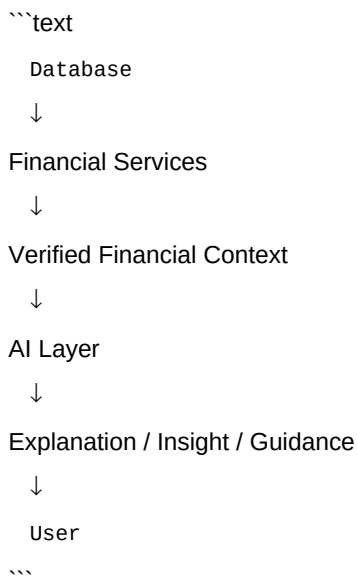
Forecasts must:

- show assumptions where useful
- communicate uncertainty
- avoid presenting estimates as facts
- use verified historical/application data

13. AI Financial Coach

The AI coach should sit above the application's verified financial data.

Architecture:



The AI should not query the database directly without controlled application logic.

The application should decide what information is appropriate to provide to the AI.

AI use cases may include:

- monthly financial summaries
- spending explanations
- identifying possible spending patterns
- explaining budget changes
- generating review questions

- contextual educational guidance
- explaining financial metrics in simple language

Avoid:

- fabricated financial numbers
- unsupported claims
- guaranteed investment or financial outcomes
- pretending uncertain predictions are certain

14. Alerts

The product should eventually provide useful alerts.

Examples:

- budget approaching limit
- unusual spending
- upcoming bill
- low projected balance
- missed savings target
- recurring expense increase
- monthly spending significantly above plan

Alerts should be useful rather than noisy.

Do not notify users for every small change.

15. Bank / Statement Integration

Bank integration is a later-stage capability.

Potential sources may include:

- bank APIs
- open banking providers
- statement uploads
- CSV imports

The integration layer must be designed independently from the core transaction domain where possible.

Imported transactions should go through:

```text

Import

↓

Validation



Normalization



Duplicate Detection



Categorization



Transaction Storage

...

Do not assume that bank integrations will always provide perfectly formatted transaction data.

## 16. Personalization

The product should eventually learn useful financial patterns from a user's historical activity.

Potential insights:

- recurring spending patterns
- category trends
- spending changes
- savings consistency
- budget adherence
- common overspending periods

Personalization must be based on actual user data.

Do not invent user behavior.

## 17. Frontend Architecture

The frontend should remain modular.

A conceptual structure may look like:

```
```text
```

```
src/
```

```
■■■■ components/
```

```
■■■■ pages/
```

```
■■■■ features/
```

```
■ ■■■ transactions/
```

```
■ ■■■ budgets/
```

```
■ ■■■ goals/
```

```
■ ■■■ bills/
■ ■■■ planning/
■ ■■■ insights/
■ ■■■ dashboard/
■■■ hooks/
■■■ services/
■■■ lib/
■■■ types/
■■■ utils/
...

```

The exact structure must follow the existing repository.

Do not restructure the entire project merely to match this example.

18. Backend Architecture

A conceptual backend flow:

```
```text
Routes
 ↓
Controllers
 ↓
Services
 ↓
Repositories / Database
...

```

Business logic should primarily live in services or appropriate domain modules rather than route handlers.

The exact architecture should follow the existing backend.

## 19. Financial Calculation Layer

Financial calculations should have centralized logic.

Examples:

```
```text
calculateBalance()
calculateMonthlyIncome()
calculateMonthlyExpenses()
calculateBudgetRemaining()

```

```
calculateSavingsProgress()  
calculateProjectedSpending()  
calculateSafeToSpend()  
````
```

Do not duplicate these formulas across:

- React components
- controllers
- database queries
- AI prompts

Where possible, calculations should have automated tests.

## 20. Data Integrity

Financial data must be treated as high-integrity data.

Important requirements:

- authenticated ownership
- authorization checks
- validation
- consistent monetary representation
- transaction-safe database operations where needed
- protection against duplicate operations
- careful handling of deletion and updates

Never sacrifice data integrity for implementation speed.

## 21. Security

Security requirements include:

- secure authentication
- authorization
- input validation
- protected API endpoints
- secret management
- safe database access
- protection against injection attacks
- secure handling of personal financial information

Never expose sensitive financial information belonging to another user.

Every user-facing financial query must be scoped to the authenticated user.

## 22. Dashboard

The dashboard should eventually become the user's financial command center.

Potential sections:

```text

Financial Overview

↓

Current Position

↓

Safe to Spend

↓

Monthly Plan

↓

Budget Status

↓

Upcoming Bills

↓

Goals

↓

Insights

↓

Recent Transactions

```

The exact dashboard should remain focused.

Do not display every metric simultaneously.

Prioritize information that helps the user decide what to do.

## 23. Monthly Review

A monthly review should compare:

```text

Income

Planned Spending

Actual Spending

Savings

Budget Performance

Goals

Projected Position

...

Potential questions:

- What went well?
- Where did spending exceed the plan?
- What changed?
- What should be adjusted next month?

AI can help explain the review, but the underlying numbers must come from deterministic application calculations.

24. Error and Empty States

Every major financial feature must handle:

- loading
- success
- empty state
- validation errors
- server errors
- network failures
- unauthorized access

Examples:

No transactions:

> "No transactions yet. Add your first income or expense to start tracking."

No monthly plan:

> "Create your monthly plan to see how your actual spending compares with your plan."

The exact copy can evolve with the UI.

25. Accessibility

The product should support accessible interaction.

Requirements include:

- semantic HTML
- keyboard navigation
- accessible form labels

- meaningful button labels
- appropriate focus states
- sufficient contrast
- screen-reader-friendly information

Do not communicate important financial information through color alone.

26. Responsive Design

The application should work across:

- desktop
- tablet
- mobile

Financial dashboards should not simply shrink the desktop layout.

Important information should be prioritized and reorganized appropriately on smaller screens.

27. Performance

Performance priorities:

1. Fast initial experience
2. Efficient API requests
3. Efficient database queries
4. Reasonable rendering
5. Avoid unnecessary network requests
6. Avoid unnecessary recalculation

Optimize based on actual bottlenecks.

Do not introduce complexity without evidence of a performance problem.

28. Testing Strategy

Tests should focus heavily on business-critical behavior.

Highest priority:

1. Financial calculations
2. Authentication
3. Authorization
4. Transaction operations
5. Budget calculations

6. Goal calculations
7. Safe-to-Spend calculations
8. Forecasting assumptions

Test edge cases.

Financial calculations should not rely solely on manual testing.

29. Observability

As the application grows, consider:

- structured server logs
- meaningful error messages
- API monitoring
- database monitoring
- application performance monitoring

Never log sensitive financial information unnecessarily.

30. Development Roadmap

The roadmap is intentionally incremental.

Phase 1 — Foundation

- Understand existing application
- Stabilize current features
- Clean architecture where necessary
- Establish reliable financial calculations
- Improve validation and testing

Phase 2 — Planning

- Monthly financial plan
- Category budgets
- Savings targets
- Financial goals
- Bills

Phase 3 — Decision Support

- Safe-to-Spend

- Financial health
- Forecasting
- Budget warnings
- Useful alerts

Phase 4 — AI Layer

- Monthly AI summaries
- Financial explanations
- Personalized insights
- AI financial coach

Phase 5 — Financial Data Integration

- Statement import
- CSV import
- Bank/open-banking integrations where appropriate
- Transaction normalization

Phase 6 — Personalization

- Historical behavior analysis
- Spending pattern detection
- Personalized recommendations
- Long-term financial progress

Do not skip foundational phases simply to build advanced AI features.

31. Feature Development Standard

Every new feature should answer:

Problem

What user problem does this solve?

Inputs

What data does it require?

Business rules

How should it behave?

Output

What should the user see?

Edge cases

What happens when information is missing or unusual?

Security

Who can access it?

Persistence

Does it require database changes?

API

Does it require new endpoints?

UI

Where does it appear?

Testing

How will it be verified?

32. Decision Log

Important product and architecture decisions should eventually be documented.

Examples:

- why a particular database representation was selected
- how Safe-to-Spend is calculated
- how budgets work
- how recurring bills are represented
- how imported transactions are normalized
- how AI receives financial context

Do not silently introduce major business rules.

33. What This Specification Does Not Mean

This document is a product direction and architectural reference.

It does NOT mean:

- all features must be implemented now
- every proposed database entity must exist
- the conceptual folder structure must replace the current one
- AI must be added immediately
- bank integrations must be built immediately
- the application should be rewritten

The existing repository remains the implementation source of truth.

This document describes where the product is going.

34. Source-of-Truth Hierarchy

When instructions conflict, use this priority:

1. Security and data integrity
2. Explicit current user requirements
3. Existing working application behavior
4. AGENTS.md engineering rules
5. PROJECT_SPEC.md product direction
6. General architectural preferences

If a conflict could cause data loss or a major architectural change, stop and ask for clarification.

35. Final Product Principle

The product should not become a complicated spreadsheet with a dashboard.

It should become a system that turns financial activity into understandable information and useful decisions.

The ideal experience is:

```text

I record my money

↓

The system understands my activity

↓

I create a plan

↓

The system compares reality with the plan

↓

It identifies important changes

↓

It explains what those changes mean



It helps me decide what to review or adjust



I become more financially aware over time

...

Build toward this experience incrementally.

Do not sacrifice correctness, security, or simplicity in pursuit of advanced features.